

PART II

PROBLEM-SOLVING FUNDAMENTALS

How to think before you code

Part I gave you the tools (syntax). This Part gives you the process that turns an unfamiliar problem statement into working code — the actual skill competitive programming tests. Everything from here onward assumes you use this process, even when a chapter doesn't repeat it explicitly.

Contents of Part II

- 1. How to Think Before Coding — The Problem-Solving Framework
- 2. Reading and Deconstructing a Problem Statement
- 3. Constraints → Algorithm Selection
- 4. Brute Force First, Always
- 5. From Brute Force to Optimal — The Improvement Cycle
- 6. Edge Cases: Thinking Systematically
- 7. Testing Your Own Solution Before Submitting
- 8. Time Limits, Judges, and Verdicts
- Part II Revision, Cheat Sheet, and Practice

Chapter 1 — How to Think Before Coding

1.1 Why a framework at all?

In one sentence: almost every wrong or slow solution comes from skipping a thinking step, not from not knowing enough C++.

A beginner reads a problem, immediately starts typing code, gets stuck halfway through, realizes they misunderstood the problem, deletes everything, and starts over — often more than once. This feels like "just how coding is," but it isn't. Strong competitive programmers spend a large fraction of their time before writing any code, and it makes the actual coding fast and close to bug-free. This chapter gives you that process as a fixed checklist you run through, in order, every single time.

1.2 The 15-step framework

- 1 Understand the statement — read it twice before thinking about a solution.
- 2 Identify the input and the output precisely — what exact format goes in, what exact format must come out.
- 3 Study the constraints — they tell you the intended complexity (Chapter 3).
- 4 Create your own examples — including ones not given in the problem, especially tiny and extreme ones.
- 5 Find a brute force — the simplest, most obviously-correct approach, even if it's slow.
- 6 Calculate the brute force's complexity — and check it against the constraints.
- 7 Find the bottleneck — which specific part of the brute force is too slow?

- 8 Look for a pattern or a mathematical/structural observation that removes the bottleneck.
- 9 Choose the right data structure for the improved approach.
- 10 Choose the right algorithm or technique.
- 11 Justify (prove, even informally) why the approach is correct — not just "it feels right."
- 12 Code it.
- 13 Test on edge cases (Chapter 6) — not just the sample input.
- 14 Re-check the complexity of the final code you actually wrote.
- 15 Optimize further only if the complexity is still too slow for the constraints.

⚠ **The single most valuable rule in this chapter**

Do not start typing code until step 5 (finding a brute force) is done in your head or on paper. If you cannot describe a slow, obviously-correct approach in one or two sentences, you do not understand the problem yet — and starting to code anyway just means debugging your own confusion, in C++, which is slower than resolving it as plain thought first.

1.3 Worked example: applying all 15 steps

Problem (paraphrased): given an array of n integers, find the maximum sum of any contiguous subarray. Constraints: $1 \leq n \leq 2 \times 10^5$, values can be negative, up to 10^9 in magnitude.

- 1 Understand: a "subarray" is a contiguous run of elements (not any subset). We want the run whose sum is largest.
- 2 Input/Output: input is n and the array; output is a single integer, the maximum sum.
- 3 Constraints: n up to 2×10^5 — from the table in the Master Roadmap, this screams $O(n)$ or $O(n \log n)$; $O(n^2)$ (4×10^{10} operations) will not finish in time.

- 4 Examples: $[1, -2, 3, 4] \rightarrow$ best is $[3, 4] = 7$. All-negative array $[-5, -1, -3] \rightarrow$ best is $[-1] = -1$ (you must pick at least one element).
- 5 Brute force: try every pair of start and end index, sum each subarray, keep the maximum.
- 6 Brute-force complexity: $O(n^2)$ pairs (or $O(n^3)$ if you re-sum from scratch each time instead of extending a running sum) — too slow for $n = 2 \times 10^5$.
- 7 Bottleneck: we are recomputing overlapping sums from scratch instead of reusing work already done.
- 8 Observation: if the sum ending at the previous position was negative, dragging it into the next subarray only hurts — better to start fresh from the current element.
- 9 Data structure: none needed beyond two plain variables (this becomes clear once the algorithm is chosen).
- 10 Algorithm: this observation is exactly Kadane's algorithm — a linear scan keeping a running "best sum ending here."
- 11 Justification: at every position, the best subarray ending there either extends the previous best-ending-here sum (if positive) or restarts at the current element (if the previous sum was negative and would only drag the total down) — this covers every possible subarray exactly once.
- 12 Code: shown below.
- 13 Edge cases: all-negative array, single-element array, array with one very large negative followed by large positives.
- 14 Final complexity: $O(n)$ time, $O(1)$ extra space — fits the constraints comfortably.
- 15 Optimize further? Not needed — $O(n)$ is already optimal for this problem.

```
long long maxSubarraySum(vector<int> &a) {
    long long bestEndingHere = a[0];
    long long best = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        bestEndingHere = max((long long)a[i], bestEndingHere +
a[i]);
        best = max(best, bestEndingHere);
    }
    return best;
}
```

Kadane's algorithm — arrived at through the 15-step process, not memorized in isolation.

□ The point of this worked example

Notice that the algorithm itself (Kadane's) is only step 11 of 15. Everything before it is what actually produces the idea — most students who "don't know DP/greedy tricks" are really missing steps 5–9, not the trick itself.

Chapter 2 — Reading and Deconstructing a Problem Statement

2.1 The four things every statement gives you

- Story — flavour text; almost always irrelevant to the algorithm, but defines the objects (array, graph, string, grid...) the problem is really about.
- Input format — the exact shape of what you will read (Chapter 4, Part I patterns apply directly).
- Output format — the exact shape of what you must print, including edge-case outputs like "print -1 if impossible."

- Constraints — numeric bounds on every input quantity; the single most decision-relevant part of the statement (Chapter 3).

⚠ **Story ≠ substance**

A problem about "distributing candies to children" and a problem about "partitioning an array so each part's sum is under a limit" can be the literal same underlying algorithm. Strip the story away mentally and restate the problem in terms of arrays, graphs, or numbers before thinking about a solution — this alone prevents a lot of wasted time.

2.2 Translating a story problem into a formal one

Story version: "There are n stations in a line. You start with k units of fuel. Station i gives you $\text{fuel}[i]$ extra units when you pass it but costs $\text{cost}[i]$ to travel to the next station. Can you complete a full loop starting from some station?"

Stripped, formal version: given two arrays $\text{fuel}[]$ and $\text{cost}[]$ of length n , does there exist a starting index such that, moving around the array circularly and maintaining a running total ($\text{fuel}[i] - \text{cost}[i]$), the running total never goes negative?

The formal version is what you actually reason about and code. Practicing this translation explicitly — writing the formal restatement in your own words before touching code — is one of the fastest ways to stop misreading problems under time pressure.

2.3 Questions to ask of every statement

- Can values repeat? Can they be negative? Can they be zero?
- Is the array/string 0-indexed or 1-indexed in the problem's own explanation?
- Is there always a valid answer, or must you detect and report "impossible"?

- Are there multiple test cases, and does the total size across all test cases matter (a "sum of n over all test cases" constraint), not just the size of a single test case?
- Is the answer required exactly, or modulo some number (a strong hint the answer can be astronomically large)?

□ "Sum of n over all test cases"

This exact phrase in a constraints section means: even though a single test case might have n up to 10^5 , the total work across every test case in one input file is bounded by that same 10^5 (or a stated separate bound). It changes what "fast enough" means — always read it, it is easy to miss.

Chapter 3 — Constraints → Algorithm Selection

3.1 Why constraints are a message from the problem-setter

Problem-setters choose constraints deliberately so that the intended solution passes and the "obvious but too slow" solution does not. This means the constraints are effectively a hint about the required complexity — reading them carefully, before designing an approach, tells you what kind of algorithm to even look for.

A standard judge executes roughly 10^8 to 10^9 simple operations per second, and a typical time limit is 1–2 seconds. That gives you a rough total operation budget of about 10^8 – 10^9 . Divide that budget by n (or n^2 , or 2^n , etc.) to sanity-check an approach before implementing it.

3.2 The constraints-to-complexity table

n up to...	Likely intended complexity	Typical technique
10	$O(n!)$ or full exhaustive search	Permutations, brute-force recursion
20	$O(2^n \cdot n)$ or $O(2^n)$	Bitmask subsets, bitmask DP
40	$O(2^{(n/2)})$	Meet in the middle
100–500	$O(n^3)$	Floyd-Warshall, simple 3D DP
2,000–10,000	$O(n^2)$ or $O(n^2 \log n)$	Simple DP, nested loops, some sorts
10^5 – 2×10^5	$O(n \log n)$	Sorting, binary search, heaps, segment tree, DSU
10^6 – 10^7	$O(n)$ or $O(n \log n)$ with a small constant	Sieve, prefix sums, two pointers, linear DP
10^8 and beyond	$O(n)$ with very light per-element work, or $O(\log n)$ per query	Bitwise tricks, math formulas, offline processing

⚠ These are guidelines, not laws

A well-optimized $O(n^2)$ with a tiny constant factor can beat a poorly-implemented $O(n \log n)$ with a huge constant factor, especially with n in the low thousands. Always treat this table as a first filter to rule out clearly-too-slow ideas quickly, not as a guarantee that anything matching the row will pass.

3.3 Multiple constraints interacting

Many problems bound two or more quantities at once — for example $n \leq 10^5$ and $q \leq 10^5$ queries. If a naive approach answers each query in $O(n)$, the total work is $O(n \cdot q) = 10^{10}$, too slow — even though n alone and q alone both look individually reasonable. Always multiply out every pair of bounds that your approach's complexity depends on before concluding it's fast enough.

Problem 3.1 — Multiply the bounds [Level 1]

A problem has $n \leq 2 \times 10^5$ elements and $q \leq 2 \times 10^5$ queries, each query asking for the sum of a range. An approach recomputes each range sum by looping over the range every time. Is this fast enough?

Key observation: Worst case, every query's range covers all n elements: total work is $O(n \cdot q) = 4 \times 10^{10}$.

Approach: Too slow. A prefix-sum array (Part III) answers each query in $O(1)$ after $O(n)$ preprocessing, giving $O(n + q)$ total — this is the pattern that should come to mind whenever you see 'many range-sum queries.'

Chapter 4 — Brute Force First, Always

4.1 Why deliberately write a slow solution

A brute force is the simplest approach you can describe that you are confident is correct, with no regard for speed. It is valuable even when you already suspect it's too slow, for three reasons:

- 1 It forces you to actually understand the problem correctly, since a brute force has nowhere to hide a misunderstanding.
- 2 It gives you a reference implementation to test a faster solution against, on small random inputs (this technique is called stress testing and is one of the most powerful debugging tools in competitive programming).
- 3 On problems with small constraints, the brute force is the intended solution — you save time by not searching for a cleverer approach that isn't needed.

4.2 Finding a brute force quickly

A reliable default: enumerate the most literal interpretation of the problem. "Find the best pair" → try every pair. "Find the best subset" → try every subset (2^n of them). "Find the shortest path" → try every path. These are almost always correct, and almost always too slow for the full constraints — which is exactly the point; you now have a starting point to optimize from (Chapter 5).

```
// Brute force for "maximum sum contiguous subarray" from Chapter 1
// - O(n^2)
long long bruteForce(vector<int> &a) {
    int n = a.size();
    long long best = a[0];
    for (int i = 0; i < n; i++) {
        long long sum = 0;
        for (int j = i; j < n; j++) {
            sum += a[j];
            best = max(best, sum);
        }
    }
    return best;
}
```

4.3 Stress testing: using the brute force to find bugs

Once you have both a brute force and a faster solution, you can catch bugs in the faster one automatically: generate many small random inputs, run both solutions on each, and check that their outputs match. This finds edge cases and logic errors far faster than manually inventing test cases.

```
// Pseudocode for a stress test loop (run locally, not submitted)
for (int test = 0; test < 1000; test++) {
    generate a small random array a (e.g. size <= 8, values in [-10,
10]);
    long long r1 = bruteForce(a);
    long long r2 = fastSolution(a);
    if (r1 != r2) {
        print the array a and both answers, then stop;
    }
}
```

□ This is a professional habit, not just a beginner's crutch

Experienced competitive programmers stress-test routinely on any problem where the fast solution isn't obviously correct by inspection — it is far quicker than manually tracing through a subtle DP transition or greedy proof looking for the one case where it breaks.

Chapter 5 — From Brute Force to Optimal: The Improvement Cycle

5.1 The five-stage cycle

This cycle — Naive → Why it's slow → Observation → Improvement → Optimal — is the actual mechanism behind most non-trivial competitive programming solutions. It will reappear explicitly throughout this book, because internalizing it is more valuable than memorizing any single algorithm.

Stage	Question you're answering
1. Naive	What is the simplest correct approach?
2. Why it's slow	Which specific operation is repeated unnecessarily, or over too large a space?
3. Observation	Is there structure (sortedness, monotonicity, overlapping subproblems, a mathematical identity) that lets us skip or combine repeated work?
4. Improvement	What data structure or technique exploits that observation?
5. Optimal	Does the new complexity fit the constraints? If not, repeat the cycle on the new bottleneck.

5.2 Worked example: Two Sum

Problem: given an array of n integers and a target T , determine whether two distinct elements sum to T .

Stage 1 — Naive

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        if (a[i] + a[j] == T) return true;
return false;
```

Stage 2 — Why it's slow

$O(n^2)$ pairs checked. For $n = 10^5$ this is $\sim 10^{10}$ operations — far too slow.

Stage 3 — Observation

For each element $a[i]$, we only need to know: has $T - a[i]$ appeared before? Checking "has X appeared" is exactly what a hash set answers in $O(1)$ average time.

Stage 4 — Improvement

```
unordered_set<int> seen;
for (int i = 0; i < n; i++) {
    if (seen.count(T - a[i])) return true;
    seen.insert(a[i]);
}
return false;
```

Stage 5 — Optimal

$O(n)$ time, $O(n)$ space — comfortably fits $n = 10^5$. No further improvement is needed; the cycle stops here.

The pattern to internalize

"I need to check a pair" almost always suggests: can I flip it into "for each element, do I need something about the elements I've already seen" — and if so, a hash set/map turns an $O(n^2)$ pair-check into an $O(n)$ single pass. This exact transformation reappears constantly across very different-looking problems.

Chapter 6 — Edge Cases: Thinking Systematically

6.1 Why the sample input is not enough

Sample inputs given in a problem statement are chosen to illustrate the problem, not to break your solution. Judges use separate, hidden test data specifically designed to catch mistakes at the boundaries. A solution that passes the sample but fails hidden tests has almost always missed an edge case, not made a conceptual error.

6.2 A systematic edge-case checklist

Category	Cases to check
Size extremes	$n = 0$ (if allowed), $n = 1$, the maximum allowed n
Value extremes	all elements equal, already sorted, reverse sorted, all zero, maximum/minimum allowed value, negative numbers if allowed
Structure extremes	duplicates present, an empty array/string, a single character string
Graph-specific	disconnected graph, self-loops, a graph with a single node, a graph that is already a cycle, multiple edges between the same pair of nodes
Numeric-specific	values that overflow <code>int</code> but not <code>long long</code> , division by a value that could be zero, floating point values needing a tolerance for comparison

⚠ **Make this checklist a habit, not an afterthought**

Run through the relevant rows of this table before submitting, every time — treat it as one more mandatory step in the framework from Chapter 1 (it lives at step 13), not an optional nicety for when you have spare time.

6.3 Worked example: edge cases for Kadane's algorithm

Revisiting the maximum-subarray-sum problem from Chapter 1:

- All negative: $[-5, -1, -3] \rightarrow$ answer must be -1 (the least negative single element), not 0 — a buggy implementation that initializes its running best to 0 would wrongly report 0 , implying "pick no elements," which the problem doesn't allow.
- Single element: $[7] \rightarrow$ answer is 7 — check the loop handles $n = 1$ without reading out of bounds.
- Large values: values near 10^9 , summed over up to 2×10^5 elements $\rightarrow$ confirms the `long long` accumulator from Part I is necessary here.

Chapter 7 — Testing Your Own Solution Before Submitting

7.1 A short pre-submission checklist

- 1 Re-read the output format — exact spacing, capitalization, and presence/absence of a trailing newline matter on some judges.
- 2 Run the provided sample input(s) and check for an exact match, not just "looks about right."
- 3 Manually construct and run at least two edge cases from the Chapter 6 checklist relevant to this problem.
- 4 Re-check the complexity of the code you actually wrote (not the complexity you intended) against the constraints — an accidental extra loop or an unnecessary copy of a large container can silently change the complexity class.
- 5 Check every variable that accumulates a sum, count, or product for potential overflow.

7.2 Reading your own code as literally as the compiler does

This connects back to Chapter 1 of Part I: when a solution gives a wrong answer, resist the urge to stare at the whole file. Instead, pick the smallest input that reproduces the wrong output and dry-run your code on it by hand, line by line, exactly as Section 1.6 (Part I) described. Most bugs reveal themselves within the first few lines of a careful dry run.

Chapter 8 — Time Limits, Judges, and Verdicts

8.1 How online judges evaluate your code

An online judge (Codeforces, LeetCode, and similar platforms) compiles your submission, then runs it against a set of hidden test files, one at a time, each under a fixed time limit and memory limit. Your submission gets a single overall verdict, usually the verdict of the first test case that fails, or "Accepted" if every test case passes.

8.2 Common verdicts and what they tell you

Verdict	What it means	What to check first
Accepted (AC)	All hidden tests passed	—
Wrong Answer (WA)	Output differs from expected on some test	Logic error or a missed edge case — revisit Chapter 6
Time Limit Exceeded (TLE)	Ran out of time on some test	Complexity too high for the constraints — revisit Chapter 3
Memory Limit Exceeded (MLE)	Used too much memory	An array/container sized too large, or unnecessary copies of big data
Runtime Error (RE)	Crashed while running	Out-of-bounds access, division by zero, stack overflow from deep recursion
Compilation Error (CE)	Code didn't compile	Syntax error — re-read the exact error message the judge shows

□ A verdict is diagnostic information, not just a grade

Each verdict points you toward a specific category of bug. Getting TLE means your logic is very likely correct but your complexity is wrong — don't go hunting for a logic bug that probably isn't there; go back to Chapter 3 and 5 instead.

Part II — Master Revision

Must Remember

- Never start coding before you can state a brute-force approach in one or two sentences.
- Constraints tell you the intended complexity — check the constraints-to-complexity table before designing an approach, not after implementing one.
- The improvement cycle is Naive → Why it's slow → Observation → Improvement → Optimal — repeat it if the first improvement still isn't fast enough.
- "For each element, do I need something about elements already seen?" is one of the most common triggers for turning an $O(n^2)$ pair-check into an $O(n)$ pass with a hash set/map.
- Multiply out every pair of bounds your complexity depends on (e.g. n and q) before concluding an approach is fast enough.
- Stress-test a fast solution against a brute force on small random inputs whenever correctness isn't obvious by inspection.
- A verdict (WA, TLE, MLE, RE) tells you which category of bug to look for — use it, don't ignore it.

Patterns to Remember

Clue in the problem	Reflex
"Find the best pair / does a pair exist"	Consider a hash set/map to avoid $O(n^2)$
"Many range-sum queries"	Consider prefix sums ($O(1)$ per query after $O(n)$ preprocessing)
$n \leq 20$	Consider bitmask enumeration of subsets
"Answer modulo 10^9+7 "	Expect the true answer to be astronomically large
"Minimum/maximum value such that..."	Consider binary search on the answer

Interview-Style Questions

- 1 Why is it valuable to write a slow, obviously-correct brute force even when you suspect it won't pass?
- 2 How do constraints tell you what complexity to aim for, and why are 10^8 - 10^9 operations per second a useful rule of thumb?
- 3 What is stress testing, and what category of bug is it best at catching?
- 4 If a submission gets Time Limit Exceeded, why is it usually not worth looking for a logic bug first?

Practice — Part II

Problem P2.1 — Design before you code [Level 1]

Problem: given an array of n integers ($n \leq 10^5$), find the length of the longest strictly increasing contiguous run. Walk through steps 1–11 of the framework (understand $\rightarrow$ justify) before writing any code.

Hint 1: What is the brute force here, and what makes it slow?

Key observation: A single left-to-right pass, extending a current run length and resetting it whenever the sequence stops increasing, visits each element once.

Complexity: $O(n)$ time, $O(1)$ extra space for the optimal approach.

Problem P2.2 — Constraints reading drill [Level 0]

A problem states $n \leq 18$ and asks for the number of ways to partition a set into groups satisfying a condition. What does $n \leq 18$ suggest about the intended approach, and why?

Approach: $n \leq 18$ to ≤ 20 is the classic signal for bitmask enumeration over subsets ($2^{18} \approx 262,144$ subsets), often combined with bitmask DP.

Problem P2.3 — Stress test design [Level 2]

You have a brute-force $O(n^2)$ solution and a suspected $O(n \log n)$ solution for the same problem. Describe a stress-testing setup that would help you find a disagreement, including what kind of random inputs to generate.

Approach: Generate many (e.g. hundreds to thousands) small random arrays, varying size from 1 to ~ 10 and values across a small range including negatives and duplicates, run both solutions on each, and stop at the first array where their outputs differ — small inputs make the disagreement easy to inspect by hand.

□ **Coming up in Part III**

Part III — Arrays and Strings applies this exact problem-solving process to the two data structures you will use in almost every problem in this book, going far beyond basic indexing into subarrays, subsequences, 2D arrays, and the string techniques that set up later chapters on hashing, KMP, and tries.